

Cost-Aware AgentRouter: A Cheaper Coding-Agent Frontier

Ziming Dong¹, Ming Chen, Nhan Huynh, Jason Thomo¹

¹Department of Computer Science, University of Victoria

Motivation — Should teams route between AI coding agents to cut cost?

Teams increasingly run several AI coding agents (Codex, Copilot, Cursor, Devin, Claude) side by side. Their subscription and usage prices differ **several-fold** — yet teams usually pick one agent out of habit, not per task.

Our question: Can we learn an offline router on observational GitHub data (AIDev, ~932k PRs) that picks the right agent per task, and *value* that policy without ever running a live A/B test?

Research Questions

- RQ1.** Do the five agents show different merge rates across task categories?
- RQ2.** How well can a regressor predict per-agent success from task/repo features, and does model capacity help?
- RQ3.** Can a cost-aware router improve the cost-quality trade-off over always-best-agent, and which signal achieves it?

Dataset & Setup

AIDev [Li et al. 2026]: 932k agentic PRs, 5 commercial agents, public GitHub.

Balanced corpus: down-sampled to **25,580 PRs** (5,116/agent) so minority agents are learnable.

Splits: 70/15/15 grouped by `repo_id` (no repo crosses splits).

Target: graded success label (merged = 1, open = 0.3, closed = 0); MAE metric.

Reduced-Leakage Features

Title and body are written by the agent *after* it acts — **post-treatment**. v2 drops the body and all body-derived features; it keeps:

- Cleaned title (agent tokens stripped)
- Repo metadata: language, log stars/forks
- `has_issue`, `task_type`, agent one-hot

Normalized Per-PR Cost

Agent	\$ / PR
Copilot	1.10
OpenAI_Codex	3.85
Cursor	3.85
Devin	4.50
Claude_Code	4.80

Method comparison is *price-invariant*: a new scenario only rescales the cost axis.

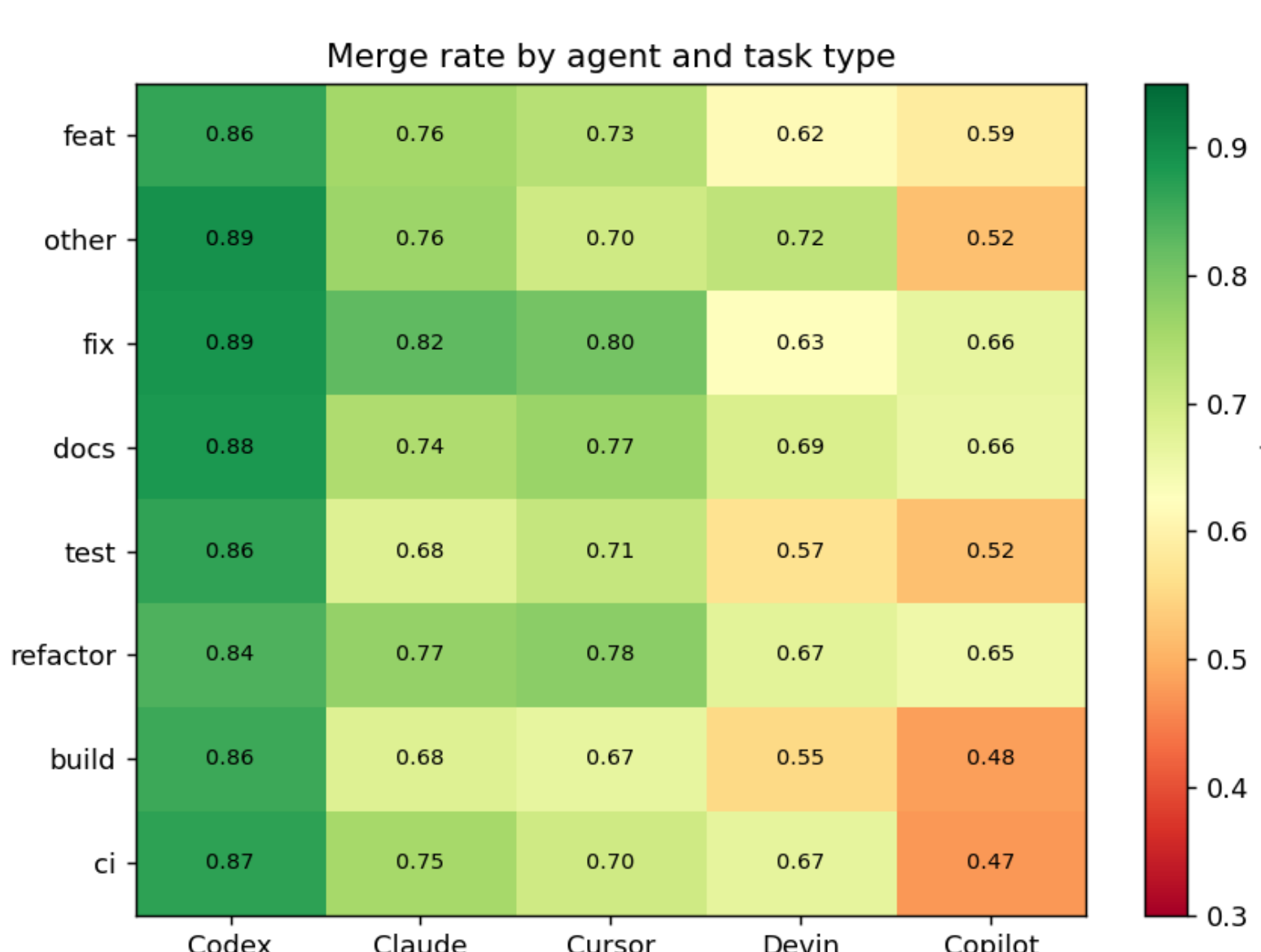
How AI Helped / Didn't

Helped: code and Latex scaffolding, figure layout, related-work search, prose drafting and around 90% code writing.

Didn't: every number comes from our own runnable pipeline, not AI text; and AI first kept the leaky PR body until we audited what the model keyed on.

Agents vs. Task Category

Codex's merge rate ranges **84-89%** against **48-66% for Copilot** across the eight most common task types: agents are not interchangeable.



But Codex leads in *every* common cell: heterogeneity is real in level yet **weak as a routing signal**.

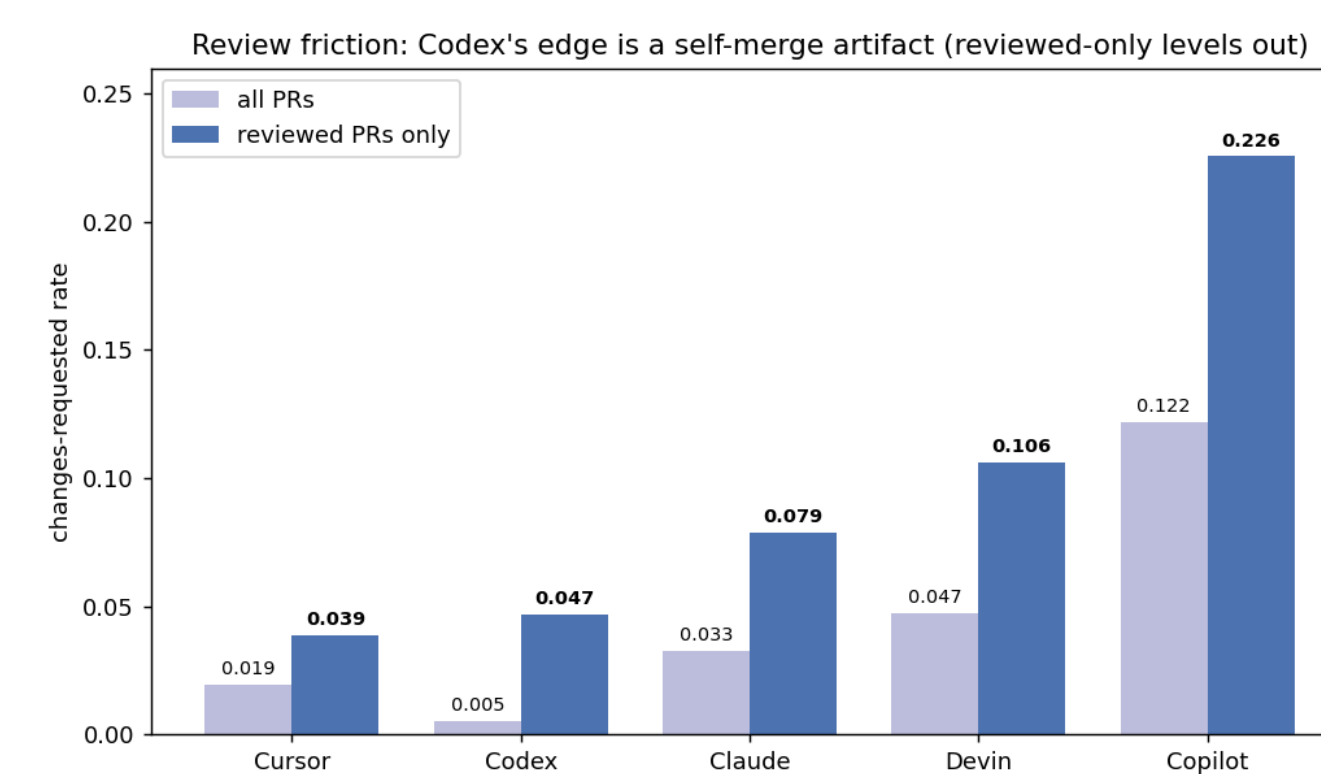
Per-Agent Success

Model (v2 title+repo features)	test MAE
per-agent-mean baseline	0.331
Ridge (linear)	0.306
XGBoost (tuned)	0.306

Linear matches XGBoost — *signal-limited*. Only **agent** and **language** carry out-of-sample signal.

Selection Bias

Propensity model predicts *acting agent* at **0.36 vs 0.20 chance**.

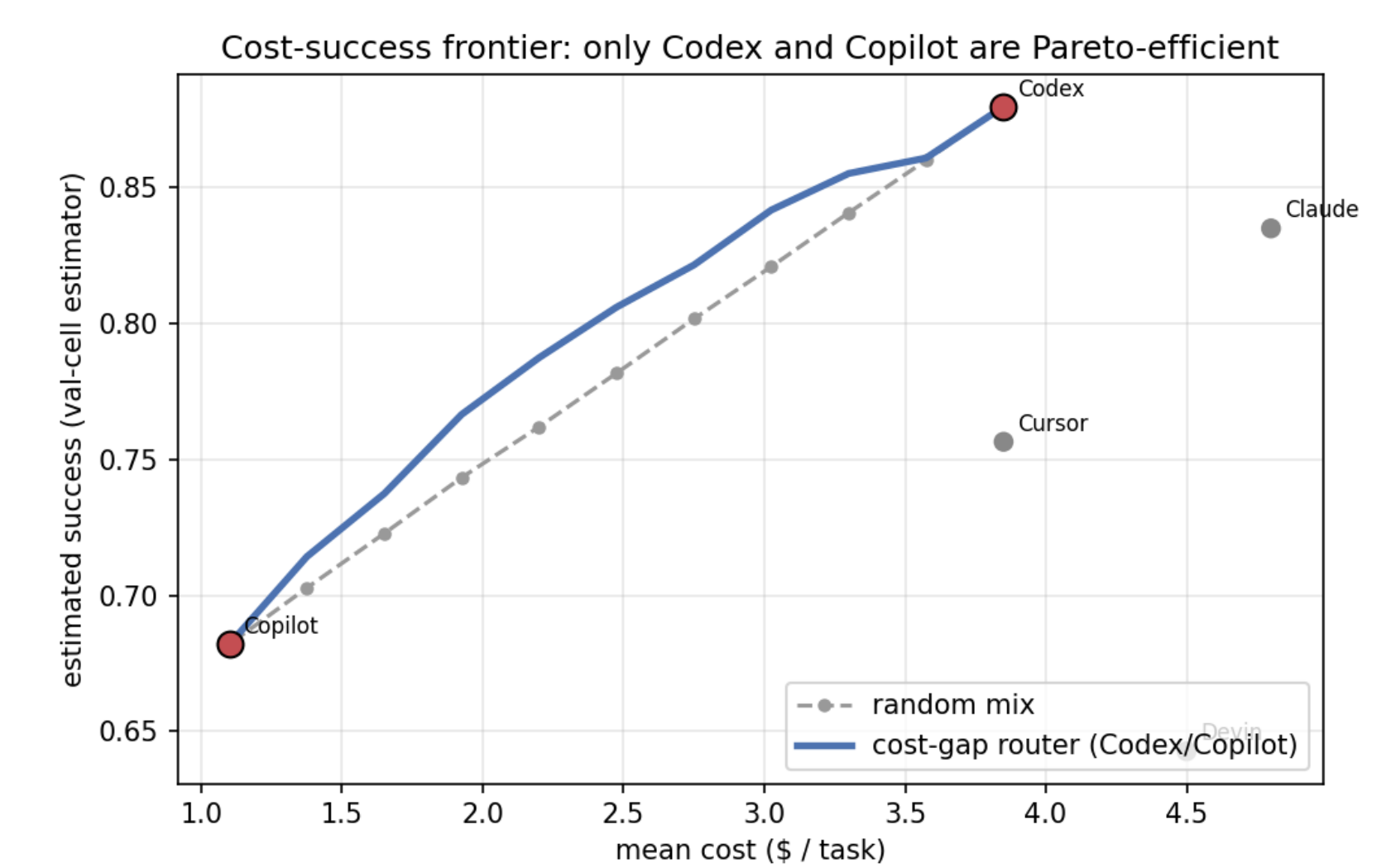


89% of Codex PRs are self-merged. Reviewed-only: Codex collapses (0.005 → 0.047); Cursor (0.039) cleaner.

What's Next

- Time-based split for temporal drift; survival label for still-open PRs.
- RouteLLM and peers as runnable baselines on AIDev (time-limited here).
- Quality labels beyond merge: reviews, reverts, tests.

A Cheaper Pareto Frontier



Under our cost scenario and the merge metric, **only Codex and Copilot** are Pareto-efficient; the cost-gap router trades along the Codex→Copilot edge. **Success at matched cost:**

Mean cost	random instance	cost-gap
\$3.85 (always-Codex)	0.880	0.880
\$3.30 (−14%)	0.841	0.855
\$2.75 (−29%)	0.801	0.821
\$2.20 (−43%)	0.761	0.787

Bottom line: keep **93% of always-Codex's success at 29% lower cost**. The edge is *significant* under direct-method and two-level bootstraps, *positive but not significant* under high-variance IPW/DR.

Key Takeaways

- Routing wins on cost, not success:** ~93% of merge rate at 29% lower cost.
- Success ranking not identifiable:** the ~4-pt estimator spread exceeds the 1–2-pt policy gaps.
- “Best” ≈ “most-used”:** always-best and most-popular pick the same agent **99%** of the time.

What We Learned

- Leakage hides in provenance.** The PR body carries agent signatures (footers, co-authored-by); mined “input” text is often written *after* the outcome.
- In-sample importance lies.** Split gain put 84% on the title, yet permutation and ablation show it carries almost no out-of-sample value.
- The mining trap.** A raw merge leaderboard reads a self-merge usage pattern as capability; adjust for assignment before trusting an observational label.

[1] H. Li, H. Zhang, A. E. Hassan. *AIDev: Studying AI Coding Agents on GitHub*. arXiv:2602.09185, 2026.
[2] I. Ong et al. *RouteLLM*. ICLR, 2025.
[3] E. Kalliamvakou et al. *Promises and Perils of Mining GitHub*. EMSE, 2016.